Creating a Species Distribution Model Using ResNet Architecture

Jackson Newell, Jayden Cozort, Reid Beckes, Md Istiak Ahammed

I. Introduction

In ecology and conservation science, Species Distribution Models (SDMs) are widely used to estimate the spatial distribution of species as a function of environmental, climatic, and geographic variables. These models aim to characterize habitat suitability and predict the probability of species presence across continuous geographic space. SDMs play a critical role in biodiversity conservation, climate change impact assessment, invasive species monitoring, and ecological planning. Accurate spatial predictions can inform conservation policies, optimize resource allocation, and improve our understanding of species–environment relationships.

Traditional SDMs have relied primarily on statistical and machine learning regression techniques such as logistic regression, random forests, and maximum entropy models. While these approaches have demonstrated strong predictive performance on structured tabular data, they often treat environmental predictors independently and may not fully exploit the spatial structure inherent in raster-based environmental datasets. With the rapid advancement of Deep Learning architectures, particularly Convolutional Neural Networks (CNNs), new opportunities have emerged to leverage spatial correlations within raster data. CNNs are specifically designed to capture local spatial patterns through convolutional operations which make them best candidate for modeling species distribution.

The goal of this paper is to develop a CNN-based Species Distribution Model that estimates the probability distribution of species presence over environmental rasters. Specifically, this study will investigate the effectiveness of a U-Net architecture and compare different optimization strategies to evaluate their impact on convergence behavior, predictive accuracy, and computational efficiency. These models predicted multiple scenarios from bird habitat suitability to a bamboo habitat investigation under the change in climate in northern China.

II. Related Work

A. Image Segmentation

Image segmentation involves the partitioning of an image into multiple different regions to better analyze the image. image segmentation has been used in many applications, from medical imaging to autonomous vehicles and face recognition. image segmentation can be split into three types: semantic, instance, and panoptic segmentation. semantic segmentation labels each pixel with a corresponding class, while instance segmentation extends the segmentation by identifying distinct instances instead, and panoptic being a combination of semantic and instance segmentation by labeling each pixel with both a class and instance ID. There are multiple different methods to perform image segmentation, these methods are: thresholding, edge-based segmentation, region-based segmentation, clustering-based segmentation, watershed, and deep learning-based segmentation. In this paper, deep learning-based segmentation will be the main method.

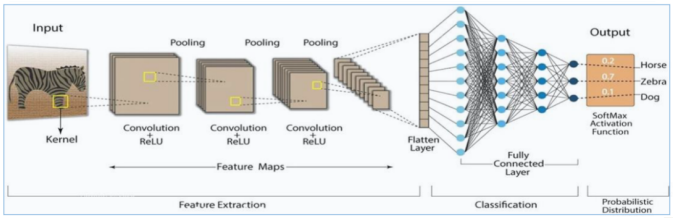

Fig. 1. Overview of CNN process

B. Convolutional Neural Networks

Convolutional Neural Networks(CNN) are a type of deep neural network that uses convolution on grid-like data to capture spatial relationships between data-points. A CNN consist of multiple layers starting with the input layer which is just the raw data in a grid form. the input layer then goes through the convolutional layers which is used to extract important features from the input. the layer applies convolution on the data, generally with a 3 by 3 kernel. It then goes through pooling layers to reduce the dimensions of the images by downsizing, making computation time faster and easing memory usage. The data is then flattened and fed into the fully connected layer that then performs classification based on the features extracted. This prediction is then sent to the output layer which is then processed using an activation function.

C. The U-Net Architecture

U-Net is a type of Convolutional Neural Network given its name from its ”U” like appearance that is mainly utilized for image segmentation. The architecture consists of three main parts: encoder, bottleneck, and decoder. The encoder uses convolution with a 3 by 3 kernel to scan the image for features followed by the application of a ReLU activation function which is then preceded by the use of max pooling with a 2 by 2 kernel to shrink the image whilst keeping important information. In a U-Net architecture the encoder is run multiple times until it reaches the bottleneck where the data is in its most compressed form and is then fed into the decoder. The decoder then uses up-sampling to increase the

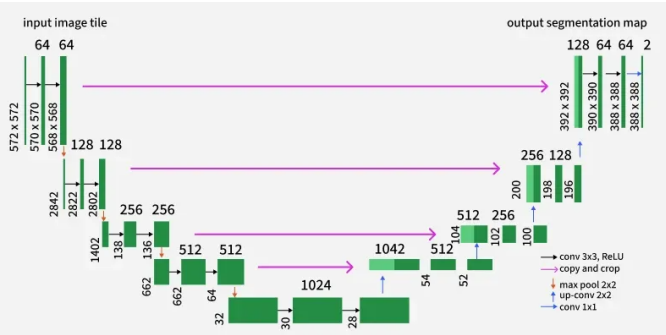

Fig. 2. Image of U-Net architecture

size of the image and counteract the shrinking done by the encoder, it then combines the information from the encoder using skip ”connections” to bring back fine detail that may have been lost from the down-sampling. The decoder then uses convolution with a 2 by 2 kernel to further refine the output. The decoder is additionally run the same number of times the encoder was ran.

III. Methodology

The first model considered was a ResNet model which would perform the classification. The original intent was to try and make a distributed heat map in this way. However, with the desired output needed, this classification method would not work. Briefly discussing the implementation, ResNet was implemented with a ResNet152 model. This means that the model has a depth of 152. This consists of many blocks of two Convolution layers and then residual connections skipping these layers as well. These skip connections help in preventing the exploding and vanishing gradient problem. This model and the UNet model were created using tensorflow tools in python. In addition, Google Colab was used so that the T4 GPU runtime could be used for more efficient performance. The code for the ResNet model was taken from a tutorial on the website ’GeeksforGeeks’ [1]. The tutorial uses repeated blocks of two convolution layers and a residual layer. Then it calculates the number of blocks needed based on the depth. In this case, the depth is 152 and has 25 of these blocks. The number of blocks is based on equation 1 below.

$$depth = (block_number * 6) + 2 \tag{1}$$

The UNet model was made using a tutorial on the same website as the ResNet tutorial [2]. The UNet model is in the same notebook as the ResNet model to keep the models in the same place for now. The implementation uses functions to define the encoders and decoders of the network. Then, it uses repeated calls to both to construct the model. The ’bottleneck’ is created by defining two convolution layers with activation layers after each. The output layer is another convolution layer with a ’sigmoid’ activation. This model is successful in building and compiling. However, due to time constraints, the model was not trained and tested on other data. The expected hurdles with the current model include loading the data to be trained on, and then making sure to change the input shape of the model to match. The full summary of the model is in the notebook in the submitted repository.

IV. Experimental Results

Currently, the required data has not been properly formatted to train the UNet architecture. Our input data are raster files, which are similar to images in that they are pixel grids that contain a value. This value corresponds to the data the raster captures instead of a color value like in an image. However, this data can be processed identically to a image in terms of applying convolution al filters. The main issue arises in combining the data. Images typically have 3 color channels. To combine enough regular raster data into one raster file, it would need to have additional channels. Our group is still exploring options to optimally combine the raster data without having a large amount of channels. Many channels may be needed which would increase training time.

Another issue our group is exploring is the output. SDMs are probability distributions over the raster pixel grid. UNet outputs binary classification of the pixels. To solve this issue, our potential solution is to remove the 1x1 convolution layer at the end of the architecture and have it output its raw probability distribution.

Our plan going forward is to create a raster of a specific species. Currently, this is Osprey as it mirrors the project of our contact in the CFRES department and allows us to compare models. An entire raster will be created for the US. Once the correct data is collected into one raster, the model will be trained on small subsets of the raster. This will be combined with data augmentation to flesh out an entire trainable dataset. Validation will be tested against a typical Maxent distribution.

V. Discussion and Conclusion

Due to a lack of ability to test results in present, no discussion or conclusion can be finalized. Next steps for the project will be mapped out instead. next steps will be training and testing, the input data will be used to train the model. Using the trained model, a probability heat map will be created of the area and be compared to more conventional methods used in the industry such as MaxEnt and Random Forest.

References

[1] GeeksforGeeks, *Residual networks (resnet) - deep learning*, Jan. 2026. [Online]. Available: https://www.geeksforgeeks.org/deep-learning/residual-networks-resnet-deep-learning/.

[2] GeeksforGeeks, *U-net architecture explained*, Oct. 2025. [Online]. Available: https://www.geeksforgeeks.org/machine-learning/u-net-architecture-explained/.